

Universidade Técnica Do Atlântico

Instituto de Engenharias e Ciências do Mar

Sidnei Cruz
scruz@uta.cv



Resumo da aula

- Algoritmos de Ordenação
 - Selection Sort
 - Insertion Sort
 - Bubble Sort

Algoritmos de Ordenação

- A existência de conjuntos de dados ordenados é bastante comum na nossa vida quotidiana e por conseguinte também o será nos sistemas informáticos.

Algoritmos de Ordenação

- A existência de conjuntos de dados ordenados é bastante comum na nossa vida quotidiana e por conseguinte também o será nos sistemas informáticos.
- Exemplos de conjuntos ordenados:

Algoritmos de Ordenação

- A existência de conjuntos de dados ordenados é bastante comum na nossa vida quotidiana e por conseguinte também o será nos sistemas informáticos.
- Exemplos de conjuntos ordenados:
 - Listas telefónicas;

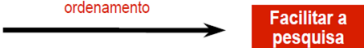
Algoritmos de Ordenação

- A existência de conjuntos de dados ordenados é bastante comum na nossa vida quotidiana e por conseguinte também o será nos sistemas informáticos.
- Exemplos de conjuntos ordenados:
 - Listas telefónicas;
 - Pautas de alunos;

Algoritmos de Ordenação

- A existência de conjuntos de dados ordenados é bastante comum na nossa vida quotidiana e por conseguinte também o será nos sistemas informáticos.
- Exemplos de conjuntos ordenados:
 - Listas telefónicas;
 - Pautas de alunos;
 - Um dicionário.

Algoritmos de Ordenação

- A existência de conjuntos de dados ordenados é bastante comum na nossa vida quotidiana e por conseguinte também o será nos sistemas informáticos.
 - Exemplos de conjuntos ordenados:
 - Listas telefónicas;
 - Pautas de alunos;
 - Um dicionário.
- Objectivo do ordenamento
- 
- Facilitar a pesquisa**
- É fundamental existirem algoritmos eficientes para ordenar grandes quantidades de dados.

Algoritmos de Ordenação

- Os algoritmos de ordenação são classificados em duas categorias:
 - Ordenação interna** - Quando o algoritmo processa dados armazenados na memória principal (pequena, rápida e de acesso aleatório). A ordenação interna de vetores corresponde, por exemplo, a ordenar um conjunto de pesos pelo seu número, onde cada um dos pesos é, individualmente, visível e acessível.

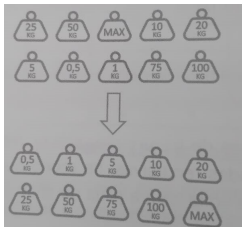


Figura: Procura Interna

Algoritmos de Ordenação

- Os algoritmos de ordenação são classificados em duas categorias:
 - Ordenação externa** - Quando o algoritmo processa dados armazenados na memória secundária (grande, lenta e de acesso sequencial). A ordenação externa de ficheiros corresponde, por exemplo, a ordenar vários baralhos de cartas colocados em diversas pilhas em cima de uma mesa, onde em cada uma das pilhas só a carta de cima é, individualmente, visível e acessível, ou uma pilha de jornais ordenados por data ou número de edição.



Figura: Procura Externa

Algoritmos de Ordenação

Algoritmo Estável

- Um algoritmo de ordenação diz-se **estável** quando preserva a ordem relativa dos elementos repetidos existentes na sequência.
- Algoritmos elementares são normalmente estáveis, mas poucos algoritmos avançados o são.

Algoritmos de Ordenação

Métodos de Ordenação

Existem algoritmos de ordenação que são simples mas pouco eficientes (complexidade quadrática), pelo que só devem ser usados para ordenar sequências de pequenas dimensões.

Estes algoritmos de ordenação simples enquadram-se num dos três **métodos** seguintes:

- Ordenação por selecção
- Ordenação por troca
- Ordenação por inserção

Algoritmos de Ordenação

Métodos de Ordenação

Ordenação por selecção

Os algoritmos de ordenação por selecção utilizam a pesquisa sequencial, ou seja, aplicam a estratégia de pesquisa exaustiva. Eles fazem passagens sucessivas sobre todos os elementos de uma parte da sequência e, em cada passagem, seleccionam o elemento de menor valor de todos os elementos analisados, no caso de se pretender uma ordenação crescente, ou, em alternativa, o elemento de maior valor de todos os elementos analisados, no caso de se pretender uma ordenação decrescente, colocando esse valor nas sucessivas posições iniciais da sequência.

Existem dois algoritmos que aplicam este método: Ordenação por **Seleccção** (Selection Sort) e o algoritmo de ordenação **Heap**(Heap Sort).

Algoritmos de Ordenação

Métodos de Ordenação

Ordenação por troca

Os algoritmos de ordenação por troca também aplicam uma estratégia de força bruta. Eles fazem passagens sucessivas sobre a sequência, comparando elementos adjacentes, que, caso estejam fora de ordem, são trocados. Quando, durante uma passagem, não se tiver efectuado qualquer troca, então é sinal que os elementos consecutivos já estão ordenados, pelo que a ordenação termina.

O **Bubble** (Bubble Sort) - aplica este método de ordenação.

Algoritmos de Ordenação

Métodos de Ordenação

Ordenação por inserção

Os algoritmos de ordenação por inserção aplicam uma estratégia de decrementar para conquistar. Eles determinam, para cada elemento da sequência, a sua posição de inserção para que a sequência fique ordenada, deslocando para o efeito os restantes elementos.

Durante o processo de ordenação, a sequência encontra-se dividida em duas partes: a parte ordenada e a restante parte da sequência com elementos ainda não processados. Cada elemento ainda por ordenar é deslocado para a parte ordenada até ser colocado na posição correta.

Existem dois algoritmos que aplicam este método: **Inserção** (Insertion Sort) e o de **Shell** (Shell Sort).



Algoritmos de Ordenação

Métodos de Ordenação

Existem algoritmos que são muito eficientes e que podem ser usados para ordenar sequências de grandes dimensões. Estes algoritmos complexos aplicam a estratégia de dividir e conquistar de forma recursiva e enquadram-se num dos dois métodos seguintes: **ordenação por fusão** e **ordenação por separação**. A estratégia de dividir para conquistar ordena uma sequência, aplicando sucessivamente de forma recursiva a partição da sequência em duas sequências parciais até que tenham apenas um elemento.

Ordenação por fusão - algoritmo **Merge** (Merge Sort)

Ordenação por separação - algoritmo **Quick** (Quick Sort)



Algoritmos de Ordenação

Porquê estudar algoritmos elementares ?

- Razões de ordem prática
 - Fáceis de codificar e por vezes suficientes
 - Rápidos/eficientes para problemas de dimensão média e por vezes os melhores em certas situações
- Razões pedagógicas
 - Bom exemplo para aprender terminologia e contexto dos problemas a codificar e por vezes suficiente
 - Alguns são fáceis de generalizar para algoritmos mais eficientes ou para melhorar o desempenho de outros algoritmos

Selection Sort

- Procedimento: Selection Sort
 1. Encontre o **menor** valor da lista/vector
 2. Troque-o com o valor na **primeira** posição
 3. Repita os passos acima para o **restante** da lista (iniciando na segunda posição e assim avançando a cada iteração)
- Na prática a sequência é dividida em **duas partes**:
 - A subsequência de itens já ordenados, a qual é construída da esquerda para direita e localiza-se no início;
 - A subsequência de itens que precisam ser ordenados, ocupando o restante do vector.

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A;

5	2	4	6	1	3
---	---	---	---	---	---

Menor: 1

1	2	4	6	5	3
---	---	---	---	---	---

Menor: 1

1	2	4	6	5	3
---	---	---	---	---	---

1	2	4	6	5	3
---	---	---	---	---	---

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #2 ⇒ A:

1	2	4	6	5	3
---	---	---	---	---	---

Menor: 2

1	2	4	6	5	3
---	---	---	---	---	---

1	2	4	6	5	3
---	---	---	---	---	---

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #3 ⇒ A:

1	2	4	6	5	3
---	---	---	---	---	---

Menor: 3

1	2	3	6	5	4
---	---	---	---	---	---

Menor: 3

1	2	3	6	5	4
---	---	---	---	---	---

1	2	3	6	5	4
---	---	---	---	---	---

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #4 ⇒ A:

1	2	3	6	5	4
---	---	---	---	---	---

Menor: 4

1	2	3	4	5	6
---	---	---	---	---	---

Menor: 4

1	2	3	4	5	6
---	---	---	---	---	---

1	2	3	4	5	6
---	---	---	---	---	---

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #5 $\Rightarrow$ A:

1	2	3	4	5	6
---	---	---	---	---	---

Menor: 5

1	2	3	4	5	6
---	---	---	---	---	---

Menor: 5

1	2	3	4	5	6
---	---	---	---	---	---

Selection Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

1	2	3	4	5	6	Menor:	6
---	---	---	---	---	---	--------	---

Iteração #6 $\Rightarrow$ A:

1	2	3	4	5	6	Menor:	6
---	---	---	---	---	---	--------	---

1	2	3	4	5	6
---	---	---	---	---	---

Selection Sort

Selection Sort

```
void selection(Item a[], int l, int r){  
    int i, j;  
  
    for (i = l; i < r; i++) {  
        int menor = i;  
        for (j = i+1; j <= r; j++)  
            if (less(a[j], a[menor]))  
                menor = j;  
        exch(a[i], a[menor]);  
    }  
}
```

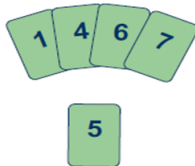
Insertion Sort

- Ordenação a partir de uma estrutura **ordenada** ou **não-ordenada**.
- Separa a estrutura a ser ordenada em duas partes: **já ordenada** e **não-ordenada**.
- Procedimento:
 - Para todo item da parte não-ordenada faça:
 - Procurar sua (nova) posição na parte já ordenada;
 - Inserir o item na posição apropriada na parte ordenada.
- Note que para **vetores**, a procura também envolve **deslocar** elementos para abrir espaço para inserção.
- O número de total de movimentos depende da desordem que reina no vector.

Insertion Sort

- Funciona de forma idêntica à ordenação das cartas na nossa mão num qualquer jogo de cartas.

Para colocar uma nova carta na posição correcta temos que afastar as outras para arranjar espaço para ela.



Insertion Sort

- Funciona de forma idêntica à ordenação das cartas na nossa mão num qualquer jogo de cartas.

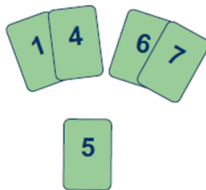
Para colocar uma nova carta na posição correcta temos que afastar as outras para arranjar espaço para ela.



Insertion Sort

- Funciona de forma idêntica à ordenação das cartas na nossa mão num qualquer jogo de cartas.

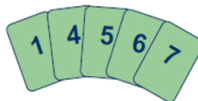
Para colocar uma nova carta na posição correcta temos que afastar as outras para arranjar espaço para ela.



Insertion Sort

- Funciona de forma idêntica à ordenação das cartas na nossa mão num qualquer jogo de cartas.

Para colocar uma nova carta na posição correcta temos que afastar as outras para arranjar espaço para ela.



Insertion Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Insertion Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Insertion Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A:

5

2	4	6	1	3
---	---	---	---	---

Iteração #2 $\Rightarrow$ A:

2	5
---	---

4	6	1	3
---	---	---	---

Insertion Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #2 $\Rightarrow$ A:

2	5	4	6	1	3
---	---	---	---	---	---

Iteração #3 $\Rightarrow$ A:

2	4	5	6	1	3
---	---	---	---	---	---

Insertion Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #2 $\Rightarrow$ A:

2	5	4	6	1	3
---	---	---	---	---	---

Iteração #3 $\Rightarrow$ A:

2	4	5	6	1	3
---	---	---	---	---	---

Iteração #4 $\Rightarrow$ A:

2	4	5	6	1	3
---	---	---	---	---	---

Insertion Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #2 $\Rightarrow$ A:

2	5	4	6	1	3
---	---	---	---	---	---

Iteração #3 $\Rightarrow$ A:

2	4	5	6	1	3
---	---	---	---	---	---

Iteração #4 $\Rightarrow$ A:

2	4	5	6	1	3
---	---	---	---	---	---

Iteração #5 $\Rightarrow$ A:

1	2	4	5	6	3
---	---	---	---	---	---

Insertion Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #2 $\Rightarrow$ A:

2	5	4	6	1	3
---	---	---	---	---	---

Iteração #3 $\Rightarrow$ A:

2	4	5	6	1	3
---	---	---	---	---	---

Iteração #4 $\Rightarrow$ A:

2	4	5	6	1	3
---	---	---	---	---	---

Iteração #5 $\Rightarrow$ A:

1	2	4	5	6	3
---	---	---	---	---	---

Iteração #6 $\Rightarrow$ A:

1	2	3	4	5	6
---	---	---	---	---	---

Insertion Sort

Insertion Sort

```
void insertion(Item a[], int l, int r) {  
    int i;  
  
    for (i = l+1; i <= r; i++)  
        compexch(a[l], a[i]);  
    for (i = l+2; i <= r; i++) {  
        int j = i;  
        Item v = a[i];  
        while (less(v, a[j-1])) {  
            a[j] = a[j-1];  
            j--;  
        }  
        a[j] = v;  
    }  
}
```


Insertion Sort

- É de fácil implementação.

Insertion Sort

- É de **fácil implementação**.
- É relativamente **eficiente** para ordenar poucos dados.

Insertion Sort

- É de **fácil implementação**.
- É relativamente **eficiente** para ordenar poucos dados.
- É **adaptativo**, ou seja, eficiente para dados ordenados.

Insertion Sort

- É de **fácil implementação**.
- É relativamente **eficiente** para ordenar poucos dados.
- É **adaptativo**, ou seja, eficiente para dados ordenados.
- É **estável**, ou seja, não altera a ordem relativa de elementos com chaves repetidas.

Insertion Sort

- É de **fácil implementação**.
- É relativamente **eficiente** para ordenar poucos dados.
- É **adaptativo**, ou seja, eficiente para dados ordenados.
- É **estável**, ou seja, não altera a ordem relativa de elementos com chaves repetidas.
- É **online**, ou seja, pode ordenar uma lista à medida que a recebe.

Bubble Sort

- Em linhas gerais, o algoritmo funciona com a seguinte filosofia:
 - Repetidamente **percorre** a lista a ser ordenada, **comparando** cada par de itens adjacentes, **trocando-os** se estiverem na ordem errada;
 - «**Empurra**» o maior elemento para o fim da sequência
 - Percorre a lista até que **nenhuma troca seja mais necessária**, o que indica que a lista está **ordenada**.

Bubble Sort

- O nome "**bolha**" deriva do fato de que os menores (maiores) elementos "**borbulham**" rapidamente para a frente (final) da lista.
- É possível que a sequência esteja ordenada antes desta ser totalmente percorrida.
 - Quando em uma passada não for efectuada nenhuma troca.

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #0 $\Rightarrow$ A:

5	2	4	6	1	3
---	---	---	---	---	---

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

5	2	4	6	1	3
---	---	---	---	---	---

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

5	2	4	6	1	3
2	5	4	6	1	3

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

5	2	4	6	1	3
2	5	4	6	1	3
2	5	4	6	1	3

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

5	2	4	6	1	3
2	5	4	6	1	3
2	5	4	6	1	3
2	4	5	6	1	3

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

5	2	4	6	1	3
2	5	4	6	1	3
2	5	4	6	1	3
2	4	5	6	1	3
2	4	5	6	1	3

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

2	4	5	6	1	3
---	---	---	---	---	---

Diagram illustrating the first iteration of Bubble Sort. The array is [2, 4, 5, 6, 1, 3]. The element 6 is highlighted in red, and the element 1 is highlighted in orange. Arrows indicate the swap between 6 and 1, resulting in the array [2, 4, 5, 1, 6, 3].

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

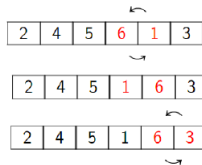
2	4	5	6	1	3
			↩		
2	4	5	1	6	3

Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A

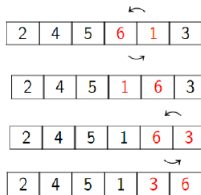


Bubble Sort - Exemplo

- Considere o vector A:

5	2	4	6	1	3
---	---	---	---	---	---

Iteração #1 $\Rightarrow$ A



Bubble Sort - Exemplo

- Ao termino da **iteração #1**:

2	4	5	1	3	6
---	---	---	---	---	---

Iteração #2 ⇒ A

2	4	5	1	3	6
---	---	---	---	---	---

Bubble Sort - Exemplo

- Ao termino da **iteração #1**:

2	4	5	1	3	6
---	---	---	---	---	---

Iteração #2 $\Rightarrow$ A

2	4	5	1	3	6
---	---	---	---	---	---

2	4	5	1	3	6
---	---	---	---	---	---

Bubble Sort - Exemplo

- Ao termino da **iteração #1**:

2	4	5	1	3	6
---	---	---	---	---	---

Iteração #2 ⇒ A

2	4	5	1	3	6
2	4	5	1	3	6
2	4	5	1	3	6

↩

↩

Bubble Sort - Exemplo

- Ao termino da **iteração #1**:

2	4	5	1	3	6
---	---	---	---	---	---

Iteração #2 ⇒ A

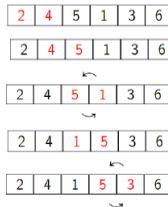
2	4	5	1	3	6
2	4	5	1	3	6
↩					
2	4	5	1	3	6
↩					
2	4	1	5	3	6

Bubble Sort - Exemplo

- Ao termino da **iteração #1**:

2	4	5	1	3	6
---	---	---	---	---	---

Iteração #2 ⇒ A

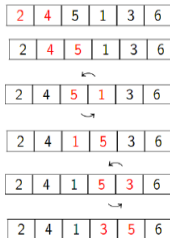


Bubble Sort - Exemplo

- Ao termino da **iteração #1**:

2	4	5	1	3	6
---	---	---	---	---	---

Iteração #2 ⇒ A



Bubble Sort - Exemplo

- Ao termino da **iteração #1**:

2	4	5	1	3	6
---	---	---	---	---	---

Iteração #2 ⇒ A

2	4	5	1	3	6
2	4	5	1	3	6
2	4	5	1	3	6
2	4	1	5	3	6
2	4	1	5	3	6
2	4	1	3	5	6
2	4	1	3	5	6

Bubble Sort - Exemplo

- Ao termino da **iteração #2**:

2	4	1	3	5	6
---	---	---	---	---	---

Iteração #3 $\Rightarrow$ A

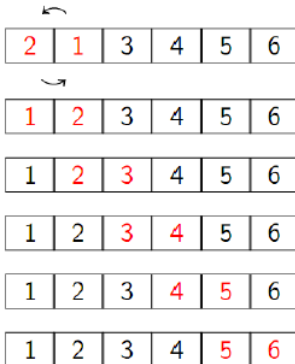


Bubble Sort - Exemplo

- Ao termino da **iteração #3**:

2	1	3	4	5	6
---	---	---	---	---	---

Iteração #4 $\Rightarrow$ A



Bubble Sort - Exemplo

- Ao termino da **iteração #4**:

1	2	3	4	5	6
---	---	---	---	---	---

Iteração #5 $\Rightarrow$ A

1	2	3	4	5	6
---	---	---	---	---	---

1	2	3	4	5	6
---	---	---	---	---	---

1	2	3	4	5	6
---	---	---	---	---	---

1	2	3	4	5	6
---	---	---	---	---	---

1	2	3	4	5	6
---	---	---	---	---	---

Sem trocas nesta iteração. **Fim do algoritmo!**

Bubble Sort

Bubble Sort (versão clássica)

```
void bubble(Item a[], int l, int r){  
    int i, j;  
  
    for (i = l; i < r; i++) /* Conta nº de passagens */  
        for (j = 0; j < r-i; j++) /* Percorre vector */  
            compexch(a[j], a[j+1]);  
}
```

Bubble Sort

Bubble Sort (versão otimizada #1)

```
void bubbleSort_Opt1(Element data[], unsigned int size){  
    unsigned int i;                //variavel auxiliar  
    int houvertroca;               //indica se houve troca  
    do{                            //percorrer novamente se houve alguma troca  
        houvertroca = 0;           //houve troca falso  
        for(i=0; i<=size-2; i++)   //percorrer ate o penultimo  
            if(data[i+1] < data[i]){  
                swap(&data[i], &data[i+1]);    //realizar a troca  
                houvertroca = 1;               //houve troca verdadeiro  
            }  
    }while(houvertroca);  
}
```

Bubble Sort

Bubble Sort (versão otimizada #2)

```
void bubbleSort_Opt2(Element data[], unsigned int size){
    unsigned int i,j;           //variaveis auxiliares
    int houvertroca;            //indica se houve troca
    j = size-2;                 //tamanho inicial do vector, -1 elemento
    do{                          //percorrer novamente se houve alguma troca
        houvertroca = 0;        //houve troca falso
        for(i=0; i<=j; i++){    //percorrer ate o penultimo "valido"
            if(data[i+1] < data[i]){
                swap(&data[i],&data[i+1]);    //realizar a troca
                houvertroca = 1;              //houve troca verdadeiro
            }
        }
        j=j-1;                  //cada iteracao temos um elemento na posicao final
    }while(houvertroca);
}
```

Algoritmos de Ordenação

Consultar os links:

<https://visualgo.net/pt>

<https://www.toptal.com/developers/sorting-algorithms>